Advanced Rust (2026): Borrow Checker, Lifetimes, Variance

Lecture 2

Lukáš Hozda

Spring 2026

MFF CUNI

Ownership and References

Ownership

- Every value has exactly one owner

Ownership

- Every value has exactly one owner
- Value is dropped when owner goes out of scope

Ownership

- Every value has exactly one owner
- Value is dropped when owner goes out of scope
- Ownership can be moved

Ownership

- Every value has exactly one owner
- Value is dropped when owner goes out of scope
- Ownership can be moved
- Borrowing is temporary non-owning access

Reference Kinds

- Shared/Immutable reference: `&T`

Reference Kinds

- Shared/Immutable reference: `&T`
  - multiple aliases allowed

Reference Kinds

- Shared/Immutable reference: `&T`
  - multiple aliases allowed
  - no mutation through this reference

Reference Kinds

- Shared/Immutable reference: `&T`
  - multiple aliases allowed
  - no mutation through this reference
- Exclusive/Mutable reference: `&mut T`

Reference Kinds

- Shared/Immutable reference: `&T`
  - multiple aliases allowed
  - no mutation through this reference
- Exclusive/Mutable reference: `&mut T`
  - unique access requirement

Reference Kinds

- Shared/Immutable reference: `&T`
  - multiple aliases allowed
  - no mutation through this reference
- Exclusive/Mutable reference: `&mut T`
  - unique access requirement
  - mutation allowed

Rules

1. A reference cannot outlive its referent
2.

Rules

1. A reference cannot outlive its referent
2. A mutable reference cannot be aliased

Rules

1. A reference cannot outlive its referent
2. A mutable reference cannot be aliased

These constraints prevent use-after-free and data races.

Example

```rust
fn as_str(data: &u32) -> &str {
    let s = format!("{}", data);
    // invalid: s is dropped at function end
    &s
}
```

Borrow checker rejects this at compile time.

Aliasing

```rust
let mut data = vec![1, 2, 3];
let x = &data[0];

data.push(4); // may reallocate
println!("{}", x);
```

Borrow checker prevents potential dangling reference.

Borrow Checker

- Tracks ownership and borrow states across control flow

- Tracks ownership and borrow states across control flow
- Validates lifetime and aliasing constraints

- Tracks ownership and borrow states across control flow
- Validates lifetime and aliasing constraints
- Rejects programs with potential memory-unsafe executions

- Tracks ownership and borrow states across control flow
- Validates lifetime and aliasing constraints
- Rejects programs with potential memory-unsafe executions
- Cost is compile-time only

- Tracks ownership and borrow states across control flow
- Validates lifetime and aliasing constraints
- Rejects programs with potential memory-unsafe executions
- Cost is compile-time only
- Paper: Stacked Borrows: An Aliasing Model for Rust

Simplified Pipeline

- Build control-flow graph

Simplified Pipeline

- Build control-flow graph
- Propagate borrow state facts per program point

Simplified Pipeline

- Build control-flow graph
- Propagate borrow state facts per program point
- Check aliasing and lifetime constraints

Simplified Pipeline

- Build control-flow graph
- Propagate borrow state facts per program point
- Check aliasing and lifetime constraints
- Emit diagnostics at violating points

NLL (Non-Lexical Lifetimes)

- Lifetime ends at last use, not always block end

NLL (Non-Lexical Lifetimes)

- Lifetime ends at last use, not always block end
- Reduces unnecessary borrow conflicts

NLL (Non-Lexical Lifetimes)

- Lifetime ends at last use, not always block end
- Reduces unnecessary borrow conflicts
- Does not relax soundness constraints

NLL (Non-Lexical Lifetimes)

- Lifetime ends at last use, not always block end
- Reduces unnecessary borrow conflicts
- Does not relax soundness constraints
- Integrated relatively recently

NLL (Non-Lexical Lifetimes)

- Lifetime ends at last use, not always block end
- Reduces unnecessary borrow conflicts
- Does not relax soundness constraints
- Integrated relatively recently
- Does not apply to Drop types

```
let mut v = vec![1, 2, 3];
let first = &v[0];
println!("{first}");

v.push(4); // allowed after last use of first
```

Reborrowing Model

- Borrow from borrow creates derived permission

Reborrowing Model

- Borrow from borrow creates derived permission
- Parent borrow is restricted while derived borrow is active

```
let mut x = 10;
let a = &mut x;
let b = &mut *a;

*b += 1;
*a += 2;
```

Lifetimes

- Lifetime is a region where reference is valid

- Lifetime is a region where reference is valid
- Lifetimes are compile-time reasoning artifacts

Explicit Lifetime Coupling

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

- Output lifetime is constrained by both inputs

Lifetime Elision

```
fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or("")
}
```

```rust
fn first_word<'a>(s: &'a str) -> &'a str {
    s.split_whitespace().next().unwrap_or("")
}
```

Elision Rules

- Each input reference gets its own lifetime

Elision Rules

- Each input reference gets its own lifetime
- If exactly one input lifetime exists, output may elide to it

Elision Rules

- Each input reference gets its own lifetime
- If exactly one input lifetime exists, output may elide to it
- For methods, `&self` lifetime can be used for output

Elision Rules

- Each input reference gets its own lifetime
- If exactly one input lifetime exists, output may elide to it
- For methods, `&self` lifetime can be used for output
- Otherwise explicit output lifetime needed

Unbounded Lifetime in Unsafe

```
fn get_str<'a>(ptr: *const String) -> &'a str {
    unsafe { &*ptr }
}
```

- This can manufacture overly-permissive lifetimes

Unbounded Lifetime in Unsafe

```
fn get_str<'a>(ptr: *const String) -> &'a str {
    unsafe { &*ptr }
}
```

- This can manufacture overly-permissive lifetimes
- Unsafe boundaries must rebind lifetime constraints immediately

`'static`

- `&'static T`: referent valid for program duration

`'static`

- `&'static T`: referent valid for program duration
- `T: 'static`: type has no non-static reference dependency

Subtyping and Variance

Lifetime Subtyping

- If `'long` fully contains `'short`, then `'long <: 'short`

Lifetime Subtyping

- If `'long` fully contains `'short`, then `'long <: 'short`
- Longer-lived reference can be used where shorter is expected

Lifetime Subtyping

```
fn debug<'a>(a: &'a str, b: &'a str) {
    println!("a = {a:?} b = {b:?}");
}

fn main() {
    let hello: &'static str = "hello";
    {
        let world = String::from("world");
        let world = &world; // 'world has a shorter lifetime than 'static
        debug(hello, world);
    }
}
```

- We're assigning `&world` to `hello` (via `input`)

- We're assigning `&world` to `hello` (via `input`)
- `&world` has a shorter lifetime than `&'static str`

- We're assigning `&world` to `hello` (via `input`)
- `&world` has a shorter lifetime than `&'static str`
- If allowed, it would create a dangling reference

- We're assigning `&world` to `hello` (via `input`)
- `&world` has a shorter lifetime than `&'static str`
- If allowed, it would create a dangling reference
- `&mut T` is invariant over `T` to prevent this exact problem

Variance Definitions

- Covariant: subtyping preserved

Variance Definitions

- Covariant: subtyping preserved
- Contravariant: subtyping reversed

Variance Definitions

- Covariant: subtyping preserved
- Contravariant: subtyping reversed
- Invariant: no subtyping relationship propagated

Common Variance Cases

- `&'a T`: covariant in `'a`, covariant in `T`

Common Variance Cases

- `&'a T`: covariant in `'a`, covariant in `T`
- `&'a mut T`: covariant in `'a`, invariant in `T`

Common Variance Cases

- `&'a T`: covariant in `'a`, covariant in `T`
- `&'a mut T`: covariant in `'a`, invariant in `T`
- `Vec<T>`: covariant in `T`

Common Variance Cases

- `&'a T`: covariant in `'a`, covariant in `T`
- `&'a mut T`: covariant in `'a`, invariant in `T`
- `Vec<T>`: covariant in `T`
- `Cell<T>`: invariant in `T`

Common Variance Cases

- `&'a T`: covariant in `'a`, covariant in `T`
- `&'a mut T`: covariant in `'a`, invariant in `T`
- `Vec<T>`: covariant in `T`
- `Cell<T>`: invariant in `T`
- `fn(T) -> U`: contravariant in `T`, covariant in `U`

Function Pointers and Contravariance

Function arguments are contravariant:

```
fn store(input: &'static str) {
    // Stores in a collection requiring 'static values
}

fn demo<'a>(input: &'a str, f: fn(&'a str)) {
    f(input);
}

fn main() {
    let local = String::from("local");
    // Error! Can't pass store (requires 'static) where
    // a function accepting any lifetime is expected
    demo(&local, store);
}
```

Borrow Splitting and Iteration

Field-Level Borrow Splitting

```
struct Point { x: i32, y: i32 }

let mut p = Point { x: 0, y: 0 };
let x = &mut p.x;
let y = &mut p.y;
*x += 10;
*y += 20;
```

Borrow checker understands disjoint struct fields.

Container Splitting Limits

```rust
let mut arr = [1, 2, 3];
let a = &mut arr[0];
let b = &mut arr[1]; // rejected
```

Index expressions are not automatically considered disjoint

Explicit Disjointness via API

```
let mut arr = [1, 2, 3, 4, 5];
let (left, right) = arr.split_at_mut(2);
left[0] += 10;
right[0] += 20;
```

Mutable Iterators

- `Iterator::next(&mut self)` can yield `&mut` items safely

Mutable Iterators

- `Iterator::next(&mut self)` can yield `&mut` items safely
- Safety relies on one-shot consumption guarantee

Mutable Iterators

- `Iterator::next(&mut self)` can yield `&mut` items safely
- Safety relies on one-shot consumption guarantee
- Iterator implementation must ensure no aliasing of yielded refs

Stacked Borrows

- Gives formal aliasing interpretation for unsafe code reasoning

- Gives formal aliasing interpretation for unsafe code reasoning
- Clarifies when pointer/reference accesses are permitted

- Gives formal aliasing interpretation for unsafe code reasoning
- Clarifies when pointer/reference accesses are permitted
- Helps explain Miri diagnostics

The Permission Stack

For each memory location:

- Permissions are tracked in a stack

The Permission Stack

For each memory location:

- Permissions are tracked in a stack
- Each new reference operation pushes to the stack

The Permission Stack

For each memory location:

- Permissions are tracked in a stack
- Each new reference operation pushes to the stack
- Creating a `&mut` invalidates all other access

The Permission Stack

For each memory location:

- Permissions are tracked in a stack
- Each new reference operation pushes to the stack
- Creating a `&mut` invalidates all other access
- When references go out of scope, they're popped

The Permission Stack

For each memory location:

- Permissions are tracked in a stack
- Each new reference operation pushes to the stack
- Creating a `&mut` invalidates all other access
- When references go out of scope, they're popped
- Accessing memory requires appropriate permission

Stacked Borrows Example

```
let mut x = 10;
let r1 = &mut x;  // Stack: [r1]
*r1 = 20;         // OK, r1 has permission
let r2 = &*r1;    // Stack: [r1, r2]
println!("{}", *r2); // OK, r2 has permission
*r1 = 30;         // Stack: [r1], r2 invalidated
// Using r2 here would be UB under stacked borrows
```

Higher-Rank Trait Bound

```rust
fn call_with_ref<F>(f: F)
where
    F: for<'a> Fn(&'a i32) -> &'a i32,
{
    let x = 10;
    let y = f(&x);
    println!("{y}");
}
```

- Useful when callback must be lifetime-polymorphic

Questions?